COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010: Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	9 / 15 / 26	**DATE SUBMITTED:**	9 / 15 / 26
NAME:	Santos, Paterno Miguel R.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Seatwork 9.1: Implementing Trees

1. PROCEDURE OUTPUT

What procedures did you perform?

main.cpp

```cpp
#include <iostream>
#include "binaryTree.h"

int main(){
    BinaryTree<char>* root = new BinaryTree<char>('A');

    std::cout << "The root is: " << root->getRootVal();
    std::cout << std::endl;

    root->insertLeft('B');
    std::cout << "The left child of root A: " << root->getLeftChild()->getRootVal() << std::endl;

    root->getLeftChild()->insertLeft('D');
    std::cout << "The left child of root B: " << root->getLeftChild()->getLeftChild()->getRootVal() << std::endl;

    root->getLeftChild()->insertRight('E');
    std::cout << "The right child of root B: " << root->getLeftChild()->getRightChild()->getRootVal() << std::endl;

    root->insertRight('C');
    std::cout << "The right child of root A: " << root->getRightChild()->getRootVal() << std::endl;

    root->getRightChild()->insertLeft('F');
    std::cout << "The left child of root C: " << root->getRightChild()->getLeftChild()->getRootVal() << std::endl;

    root->getRightChild()->insertRight('G');
    std::cout << "The right child of root C: " << root->getRightChild()->getRightChild()->getRootVal() << std::endl;

    // Setting root value
    root->setRootVal('Z');
    std::cout << "The new root is: " << root->getRootVal() << std::endl;

    // Testing isLeaf()
    std::cout << "Is A a leaf? " << root->isLeaf() << std::endl;

    std::cout << "Is D a leaf? " << root->getLeftChild()->getLeftChild()->isLeaf() << std::endl;
    return 0;
}
```

binaryTree.h

```
#ifndef BINARYTREE_H
#define BINARYTREE_H
#include <iostream>

template<class T>
class BinaryTree{
    private:
        // key
        T key;

        // pointer for left and right child.
        BinaryTree<T>* leftChild;
        BinaryTree<T>* rightChild;

    public:
        // Constructor
        BinaryTree(T rootObj);

        // Insert chidren insert left and right
        void insertLeft(T newNode);
        void insertRight(T newNode);

        // To access child nodes
        BinaryTree<T>* getLeftChild();
        BinaryTree<T>* getRightChild();

        // Get root value and set root node
        T getRootVal();
        void setRootVal(T obj);

        // Node checker if leaf
        bool isLeaf();

};
```

```cpp
template<class T>
BinaryTree<T>::BinaryTree(T rootObj){
    key = rootObj;
    leftChild = NULL;
    rightChild = NULL;
    std::cout << "Root was created \n";
}

template<class T>
T BinaryTree<T>::getRootVal(){
    return key;
}

template<class T>
void BinaryTree<T>::insertLeft(T newNode){
    // Two types

    // If currently no child
    if(leftChild == NULL){
        leftChild = new BinaryTree<T>(newNode);
    }

    // If want to insert a certain node
    else{
        // Create new node:
        BinaryTree<T>* newChild = new BinaryTree<T>(newNode);

        // Move existing left child below new node
        newChild->leftChild = leftChild;

        // Make new node as left child
        leftChild = newChild;
    }
}
```

```cpp
template<class T>
BinaryTree<T> *BinaryTree<T>::getLeftChild(){
    return leftChild;
}

template<class T>
BinaryTree<T>* BinaryTree<T>::getRightChild(){
    return rightChild;
}

template<class T>
void BinaryTree<T>::insertRight(T newNode){
    // If currently no right child
    if(rightChild == NULL){
        rightChild = new BinaryTree<T>(newNode);
    }

    // If there is already a right child
    else{
        // Create new node
        BinaryTree<T>* newChild = new BinaryTree<T>(newNode);

        // Move existing right child below new node
        newChild->rightChild = rightChild;

        // Make new node the right child
        rightChild = newChild;
    }
}
```

```
template<class T>
void BinaryTree<T>::setRootVal(T obj){
    key = obj;
}

template<class T>
bool BinaryTree<T>::isLeaf(){
    return leftChild == NULL && rightChild == NULL;
}

#endif
```

Output

```
Root was created
The root is: A
Root was created
The left child of root A: B
Root was created
The left child of root B: D
Root was created
The right child of root B: E
Root was created
The right child of root A: C
Root was created
The left child of root C: F
Root was created
The right child of root C: G
The new root is: Z
Is A a leaf? 0
Is D a leaf? 1

--------------------------------
Process exited after 1.318 seconds with return value 0
Press any key to continue . . .
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Documentation:

The BinaryTree class is used to store the left and right child node. It uses a key to store the value of each node and pointers to connect the nodes together like a linked list. The insertLeft() and insertRight() is used to add new nodes to the left or right side of the tree. The getLeftChild() and getRightChild() is also used to display the child nodes. While getRootVal() gets the value of the current node, the setRootVal() changes its value. isLeaf() checks if a node has no children and returns true when both the left and right child pointers result in null, which means the node is a leaf.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.